
MODULE *Graphs*

LOCAL INSTANCE *Naturals*
 LOCAL INSTANCE *Sequences*
 LOCAL INSTANCE *SequencesExt*
 LOCAL INSTANCE *FiniteSets*
 LOCAL INSTANCE *Relation*

$IsDirectedGraph(G) \triangleq$
 $\wedge G = [node \mapsto G.node, edge \mapsto G.edge]$
 $\wedge G.edge \subseteq (G.node \times G.node)$

$DirectedSubgraph(G) \triangleq$
 $\{H \in [node : SUBSET\ G.node, edge : SUBSET\ (G.node \times G.node)] :$
 $IsDirectedGraph(H) \wedge H.edge \subseteq G.edge\}$

$Transpose(G) \triangleq$
https://en.wikipedia.org/wiki/Transpose_graph
 $[edge \mapsto \{\langle e[2], e[1] \rangle : e \in G.edge\},$
 $node \mapsto G.node]$

$IsUndirectedGraph(G) \triangleq$
 $\wedge IsDirectedGraph(G)$
 $\wedge \forall e \in G.edge : \langle e[2], e[1] \rangle \in G.edge$

$UndirectedSubgraph(G) \triangleq \{H \in DirectedSubgraph(G) : IsUndirectedGraph(H)\}$

$Path(G) \triangleq \{p \in Seq(G.node) :$
 $\wedge p \neq \langle \rangle$
 $\wedge \forall i \in 1 \dots (Len(p) - 1) : \langle p[i], p[i + 1] \rangle \in G.edge\}$

$SimplePath(G) \triangleq$
 A simple path is a path with no repeated nodes.
 $\{p \in SeqOf(G.node, Cardinality(G.node)) :$
 $\wedge p \neq \langle \rangle$
 $\wedge Cardinality(\{p[i] : i \in DOMAIN\ p\}) = Len(p)$
 $\wedge \forall i \in 1 \dots (Len(p) - 1) : \langle p[i], p[i + 1] \rangle \in G.edge\}$

$AreConnectedIn(m, n, G) \triangleq$
 $\exists p \in SimplePath(G) : (p[1] = m) \wedge (p[Len(p)] = n)$

$ConnectionsIn(G) \triangleq$
 Compute a Boolean matrix that indicates, for each pair of nodes,
 if there exists a path that links the two nodes. The computation,
 based on *Warshall's* algorithm, is much more efficient than the
 definition used in *AreConnectedIn*, and the result can be cached
 by *TLC*, avoiding recomputation.

```

LET  $C[N \in \text{SUBSET } G.\text{node}] \triangleq$ 
  Matrix representing the existence of paths whose inner nodes
  (i.e., except for the source and the target) are all in the
  set of nodes  $N$ .
  IF  $N = \{\}$ 
  THEN  $[m, n \in G.\text{node} \mapsto m = n \vee \langle m, n \rangle \in G.\text{edge}]$ 
  ELSE LET  $u \triangleq \text{CHOOSE } u \in N : \text{TRUE}$ 
         $Cu \triangleq C[N \setminus \{u\}]$ 
        IN  $[m, n \in G.\text{node} \mapsto \vee Cu[m, n]$ 
           $\vee Cu[m, u] \wedge Cu[u, n]]$ 
  IN  $C[G.\text{node}]$ 

 $\text{IsStronglyConnected}(G) \triangleq$ 
 $\forall m, n \in G.\text{node} : \text{AreConnectedIn}(m, n, G)$ 

```

```

 $\text{IsTreeWithRoot}(G, r) \triangleq$ 
 $\wedge \text{IsDirectedGraph}(G)$ 
 $\wedge \forall e \in G.\text{edge} : \wedge e[1] \neq r$ 
 $\wedge \forall f \in G.\text{edge} : (e[1] = f[1]) \Rightarrow (e = f)$ 
 $\wedge \forall n \in G.\text{node} : \text{AreConnectedIn}(n, r, G)$ 

```

Returns the union of two graphs.

Example:

```

 $G = [\text{node} \mapsto \{1, 2\}, \text{edge} \mapsto \{\langle 1, 2 \rangle\}]$ 
 $H = [\text{node} \mapsto \{2, 3\}, \text{edge} \mapsto \{\langle 2, 3 \rangle\}]$ 
 $\text{GraphUnion}(G, H)$ 
 $= [\text{node} \mapsto \{1, 2, 3\}, \text{edge} \mapsto \{\langle 1, 2 \rangle, \langle 2, 3 \rangle\}]$ 

```

```

 $\text{GraphUnion}(G, H) \triangleq$ 
 $[\text{node} \mapsto G.\text{node} \cup H.\text{node}, \text{edge} \mapsto G.\text{edge} \cup H.\text{edge}]$ 

```

Checks whether the graph G is bipartite with partitions U and V .

Example:

```

 $G = [\text{node} \mapsto \{1, 2, 3, 4\},$ 
 $\text{edge} \mapsto \{\langle 1, 2 \rangle, \langle 2, 3 \rangle, \langle 3, 4 \rangle\}]$ 
 $\text{IsBipartiteWithPartitions}(G, \{1, 3\}, \{2, 4\}) = \text{TRUE}$ 

```

```

 $\text{IsBipartiteWithPartitions}(G, U, V) \triangleq$ 
 $\wedge U \cap V = \{\}$ 
 $\wedge G.\text{node} \subseteq (U \cup V)$ 
 $\wedge \forall e \in G.\text{edge} : \vee e[1] \in U \wedge e[2] \in V$ 
 $\vee e[2] \in U \wedge e[1] \in V$ 

```

Checks whether the graph G contains a cycle.

Note: Relies on the definition of *ConnectionsIn*. Please note that this operator is defined recursively.

```

 $\text{HasCycle}(G) \triangleq$ 

```

$$\begin{aligned}
& \vee \exists n \in G.node : \langle n, n \rangle \in G.edge \\
& \vee \exists m, n \in G.node : \\
& \quad \wedge m \neq n \\
& \quad \wedge ConnectionsIn(G)[m, n] \\
& \quad \wedge ConnectionsIn(G)[n, m]
\end{aligned}$$

Checks whether the directed graph G is a directed Acyclic Graph (DAG).

$$\begin{aligned}
IsDag(G) & \triangleq \\
& \wedge IsDirectedGraph(G) \\
& \wedge \neg HasCycle(G)
\end{aligned}$$

Returns the set of nodes that are immediate successors of node n in G .

Example: $G = [node \mapsto \{1, 2, 3\}, edge \mapsto \{\langle 1, 2 \rangle, \langle 1, 3 \rangle\}]$
 $Successors(G, 1) = \{2, 3\}$

$$Successors(G, n) \triangleq \{m \in G.node : \langle n, m \rangle \in G.edge\}$$

Returns the set of nodes that are immediate successors of any node in S .

Example: $G = [node \mapsto \{1, 2, 3\}, edge \mapsto \{\langle 1, 2 \rangle, \langle 1, 3 \rangle\}]$
 $AllSuccessors(G, \{1, 2\}) = \{2, 3\}$

$$AllSuccessors(G, S) \triangleq \text{UNION } \{Successors(G, n) : n \in S\}$$

Returns the set of nodes that are immediate predecessors of node n in G .

Example: $G = [node \mapsto \{1, 2, 3\}, edge \mapsto \{\langle 2, 1 \rangle, \langle 3, 1 \rangle\}]$
 $Predecessors(G, 1) = \{2, 3\}$

$$Predecessors(G, n) \triangleq \{m \in G.node : \langle m, n \rangle \in G.edge\}$$

Returns the set of nodes that are immediate predecessors of any node in S .

Example: $G = [node \mapsto \{1, 2, 3\}, edge \mapsto \{\langle 2, 1 \rangle, \langle 3, 1 \rangle\}]$
 $AllPredecessors(G, \{1, 2\}) = \{2, 3\}$

$$AllPredecessors(G, S) \triangleq \text{UNION } \{Predecessors(G, n) : n \in S\}$$

Returns the set of all nodes in graph G that have a path to node n .

Example: $G \triangleq [node \mapsto \{1, 2, 3, 4\}, edge \mapsto \{\langle 4, 2 \rangle, \langle 2, 1 \rangle, \langle 3, 1 \rangle\}]$
 $Ancestors(G, 1) = \{2, 3, 4\}$

$$Ancestors(G, n) \triangleq$$

$$\text{LET } EdgeRelation \triangleq$$

revert to the following syntax once *TLAPS* uses *SANY*

(<https://github.com/tlaplus/tlapm/issues/213>)

$$[(x, y) \in G.node \times G.node \mapsto \langle x, y \rangle \in G.edge]$$

$$[p \in G.node \times G.node \mapsto \langle p[1], p[2] \rangle \in G.edge]$$

$$\text{IN } \{m \in G.node : TransitiveClosure(EdgeRelation, G.node)[m, n]\}$$

Returns the set of all nodes in graph G that are reachable from node n

Example: $G \triangleq [node \mapsto \{1, 2, 3, 4\}, edge \mapsto \{\langle 4, 2 \rangle, \langle 2, 1 \rangle, \langle 3, 1 \rangle\}]$

$Descendants(G, 4) = \{1, 2\}$
 $Descendants(G, n) \triangleq$
 LET $EdgeRelation \triangleq$
 revert to the following syntax once *TLAPS* uses *SANY*
 (<https://github.com/tlaplus/tlapm/issues/213>)
 $[(x, y) \in G.node \times G.node \mapsto \langle x, y \rangle \in G.edge]$
 $[p \in G.node \times G.node \mapsto \langle p[1], p[2] \rangle \in G.edge]$
 IN $\{m \in G.node : TransitiveClosure(EdgeRelation, G.node)[n, m]\}$

Returns the in-degree of node n in directed graph G .

Example: $G = [node \mapsto \{1, 2, 3\}, edge \mapsto \{\langle 2, 1 \rangle, \langle 3, 1 \rangle\}]$
 $InDegree(G, 1) = 2$

$InDegree(G, n) \triangleq Cardinality(Predecessors(G, n))$

Returns the out-degree of node n in directed graph G .

Example: $G = [node \mapsto \{1, 2, 3\}, edge \mapsto \{\langle 1, 2 \rangle, \langle 1, 3 \rangle\}]$
 $OutDegree(G, 1) = 2$

$OutDegree(G, n) \triangleq Cardinality(Successors(G, n))$

Returns the set of root nodes of G .

$Roots(G) \triangleq \{n \in G.node : Predecessors(G, n) = \{\}\}$

Returns the set of leaf nodes of G .

$Leaves(G) \triangleq \{n \in G.node : Successors(G, n) = \{\}\}$

The graph with no nodes and no edges.

$EmptyGraph \triangleq [node \mapsto \{\}, edge \mapsto \{\}]$

The set of all possible labeled directed graphs whose node set is S .

Example:

$Graphs(\{1, 2\}) = \{$
 $[node \mapsto \{1, 2\}, edge \mapsto \{\}],$
 $[node \mapsto \{1, 2\}, edge \mapsto \{\langle 1, 2 \rangle\}],$
 $[node \mapsto \{1, 2\}, edge \mapsto \{\langle 1, 2 \rangle, \langle 2, 1 \rangle\}],$
 $\dots$
 $\}$

$Graphs(S) \triangleq [node : \{S\}, edge : \text{SUBSET } (S \times S)]$